CHAPTER 1: CONCEPTION OF THE PROJECT

1.1 Introduction

Automated video surveillance and object tracking have become essential components of modern intelligent monitoring systems. With the proliferation of IP cameras, traffic monitoring systems, and security installations, the ability to analyze video in real time is of great practical importance. Traditional software-only implementations, while flexible, often fall short of the throughput requirements imposed by real-time video processing. A hardware-accelerated approach, particularly one based on a co-design methodology combining hardware description language (HDL) logic with high-level software, offers a compelling balance between performance, flexibility, and development efficiency.

This project implements such a co-design system for real-time motion detection and multi-object tracking. The hardware processing pipeline, described in Verilog RTL, handles computationally intensive operations such as pixel-level background subtraction, buffered edge detection using Sobel filtering, and bounding box generation. The software layer, implemented in MATLAB and its Simulink environment, handles video pre-processing, co-simulation, and post-processing visualization.

1.2 Motivation

The demand for intelligent surveillance systems in urban environments, particularly for traffic monitoring, pedestrian detection, and security applications, has grown significantly. Existing commercial solutions are often either too expensive for small-scale deployment or too power-hungry for embedded scenarios. The motivation behind this project is to demonstrate a cost-effective, modular, and scalable hardware-accelerated approach to object detection and tracking that can serve as the foundational kernel for future embedded vision systems.

Additionally, from an academic standpoint, this project serves as a practical exercise in hardware-software co-design, a skill increasingly valued in the VLSI and embedded systems industry. Bridging the gap between RTL design and high-level system simulation using MATLAB Simulink mirrors real-world SoC verification and validation workflows.

1.3 Background

Object tracking in video streams is a classical computer vision problem. The fundamental pipeline involves acquiring frames, detecting regions of interest through motion or feature analysis, localizing those regions, and tracking them across successive frames. For hardware implementation, the pixel-streaming paradigm is most practical: pixels are processed one at a time in a clocked pipeline, eliminating the need for full frame buffering at the processing stage.

Sobel edge detection is a well-established gradient-based technique that computes the rate of intensity change in horizontal and vertical directions using 3x3 convolution kernels. Background subtraction is an equally foundational technique for isolating moving objects by comparing the current frame against a static or slowly updating background model. Bounding box tracking, which computes the minimum enclosing rectangle around detected edge pixels, provides a simple but effective localization mechanism.

1.4 Literature Survey

Several prior works have explored hardware acceleration for video analytics. FPGA-based Sobel edge detectors have been extensively reported, with implementations demonstrating throughput in excess of 100 million pixels per second. Line-buffer-based architectures are the standard approach for implementing 2D convolution in streaming pixel pipelines, as they avoid the cost of full-frame memory. Background subtraction on FPGAs has been explored for traffic and security applications.

MATLAB Simulink has been used as a co-simulation platform for hardware-software co-design in numerous academic projects, allowing HDL modules to be instantiated within a system-level simulation environment. This project builds upon these established techniques and integrates them into a cohesive pipeline validated through MATLAB Simulink co-simulation.

1.5 Problem Statement

Design and implement a hardware-software co-design system capable of performing real-time motion detection and multi-object tracking on a video stream. The hardware component shall be implemented as synthesizable Verilog RTL modules and shall include background subtraction, edge detection, and bounding box tracking. The software component shall use MATLAB and Simulink for co-simulation, video pre-processing, and result visualization. The system must correctly detect and annotate moving objects across multiple video frames.

1.6 Objectives

- To design a modular, synthesizable Verilog RTL pipeline for pixel-level motion detection and edge-based object tracking.
- To implement a MATLAB Simulink co-simulation environment that instantiates and drives the Verilog pipeline with real video data.
- To develop MATLAB scripts for automated video-to-pixel-stream conversion and output video reconstruction with annotated bounding boxes.
- To validate the system on a real-world traffic surveillance video and demonstrate multi-object detection capability.
- To analyze system parameters such as threshold sensitivity and minimum object area and their effect on detection accuracy.

1.7 Features

- Pipelined Verilog RTL architecture with a clock-enable (CE) control signal for gated operation.
- Configurable background subtraction threshold for motion sensitivity tuning.
- Line-buffer-based 3x3 neighborhood access enabling streaming Sobel edge computation.
- Per-frame bounding box tracking with automatic reset and coordinate latching.

- MATLAB Simulink co-simulation with automated InitFcn and StopFcn callback-driven I/O.
- Multi-object detection via MATLAB post-processing using connected component analysis (regionprops).
- Full-resolution annotated output video generation with per-frame object count overlay.

1.8 Areas of Application

- Traffic monitoring and vehicle counting systems.
- Perimeter security and intrusion detection systems.
- Pedestrian detection in smart city infrastructure.
- Industrial conveyor belt inspection and defect detection.
- Embedded vision kernels for SoC-based surveillance platforms.

1.9 Benefits

The co-design methodology employed in this project offers significant advantages over purely software-based or purely hardware-based approaches. The Verilog RTL pipeline provides a deterministic, low-latency, and highly parallel processing kernel that is suitable for eventual FPGA or ASIC implementation. The MATLAB Simulink environment provides a rich simulation, verification, and visualization platform that accelerates development and debugging. Together, they enable a faster design-to-validation cycle while producing results that are directly transferable to hardware deployment.

CHAPTER 2: DESIGN OF THE PROJECT

2.1 System Architecture Overview

The overall system is organized into two co-operating layers. The hardware layer is a synchronous, clocked, pixel-streaming pipeline implemented in Verilog RTL. The software layer consists of MATLAB scripts and a Simulink model that drive and observe the hardware pipeline. The two layers exchange data through text file intermediaries: an input pixel stream file (input_video.txt) and an output edge stream file (output_edges.txt), along with a bounding box coordinate file (box_coordinates.txt).

At the hardware level, pixels from the input stream pass through four sequential stages: background subtraction, line buffering, Sobel edge calculation, and bounding box tracking. Each module is clocked on the positive edge of the system clock and gated by a common clock-enable (CE) signal, allowing the pipeline rate to be controlled externally.

2.2 Mathematical Formulations

Background Subtraction: The absolute pixel difference between the current frame pixel (P_curr) and the background pixel (P_bg) is computed as:

$$\mathbf{diff = |P\_curr - P\_bg|}$$
$$\mathbf{is\_motion = 1 \ \ if\ diff > T\_sub,\ \ else\ 0}$$

where T_sub is the background subtraction threshold, an 8-bit configurable parameter.

Sobel Edge Detection: For a 3x3 pixel neighborhood arranged as p[row][col] (row 0 = top, row 2 = bottom), the horizontal gradient Gx and vertical gradient Gy are computed as:

$$\mathbf{Gx = (p00 + 2 \cdot p10 + p20) - (p02 + 2 \cdot p12 + p22)}$$
$$\mathbf{Gy = (p00 + 2 \cdot p01 + p02) - (p20 + 2 \cdot p21 + p22)}$$
$$\mathbf{|G| = |Gx| + |Gy| \ \ (Manhattan\ approximation)}$$
$$\mathbf{edge\_out = 0xFF \ \ if\ |G| > T\_sobel,\ \ else\ 0x00}$$

where T_sobel is an 11-bit configurable Sobel threshold. The Manhattan approximation of the gradient magnitude is used in place of the Euclidean norm to avoid the need for a hardware square root operation.

Bounding Box: The bounding box coordinates are updated per pixel. For each pixel at position (x_c, y_c) where edge_out = 0xFF:

$$\textbf{box\_min\_x} = \min(\textbf{box\_min\_x}, \textbf{x\_c}), \quad \textbf{box\_max\_x} = \max(\textbf{box\_max\_x}, \textbf{x\_c})$$
$$\textbf{box\_min\_y} = \min(\textbf{box\_min\_y}, \textbf{y\_c}), \quad \textbf{box\_max\_y} = \max(\textbf{box\_max\_y}, \textbf{y\_c})$$

Coordinates are latched to output ports at the end of each frame and reset for the next frame.

2.3 Block Diagram

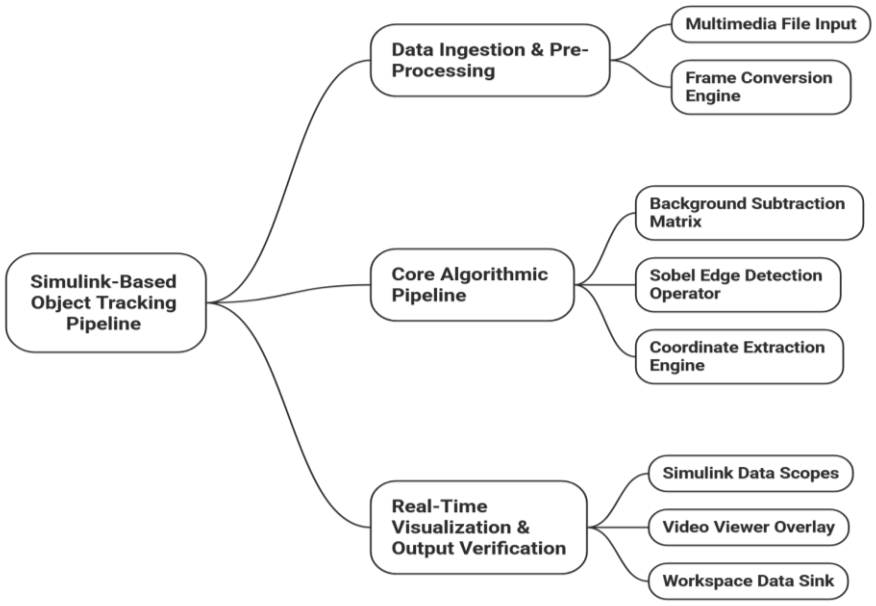

Figure 2.1: Top-Level Hardware-Software Co-Design Block Diagram

The block diagram above illustrates the data flow from the input video file through the MATLAB pre-processor, into the Simulink co-simulation environment which instantiates the Verilog RTL pipeline, and out through the MATLAB post-processor to generate the annotated output video. The hardware pipeline modules are connected in series: Background Subtraction → Line Buffer → Sobel Calculator → Tracker Box, with the top-level module (top_tracking_pipeline) providing the interconnect.

2.4 Algorithm and Flowchart

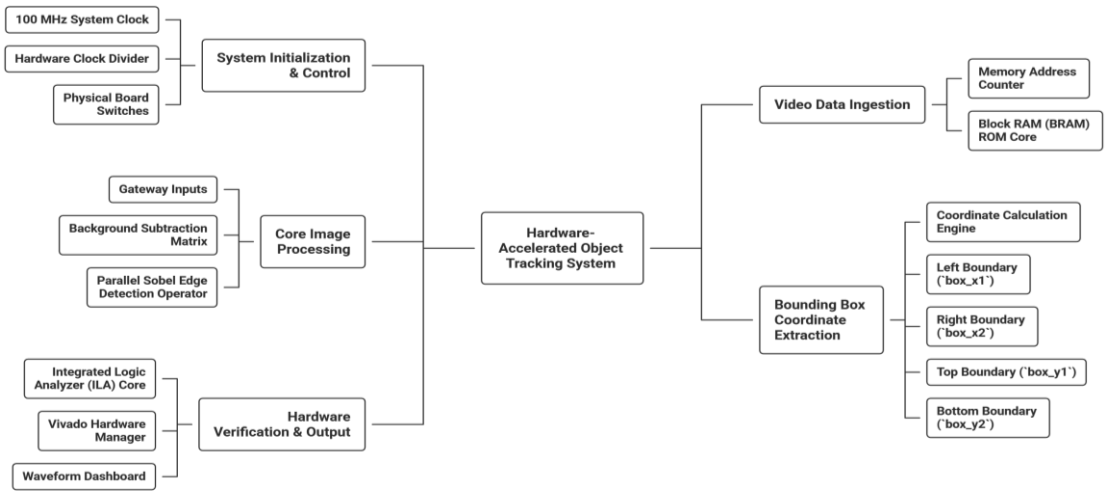

Figure 2.2: Overall System Processing Flowchart

The system flowchart above describes the complete processing sequence. The overall algorithm proceeds as follows. First, the input video is read frame by frame by the MATLAB pre-processing script, each frame is resized to 160x120 pixels and converted to grayscale, and pixel values are written as hexadecimal strings to input_video.txt. The Simulink model then reads this file through its InitFcn callback, loads the pixel stream into a workspace signal, and runs the simulation, driving the Verilog RTL pipeline one pixel per simulation timestep. Upon simulation completion, the StopFcn callback writes the output edge stream to output_edges.txt and the bounding box coordinates to box_coordinates.txt. Finally, the multi-object post-processing script reads both output files alongside the original high-resolution video, applies motion gating and morphological dilation to the hardware edge map, computes connected component bounding boxes using MATLAB's regionprops function, scales the boxes to the original resolution, and renders the annotated output video.

2.5 Module Descriptions

The hardware design consists of five Verilog modules:

background_sub

This module computes the absolute difference between the current pixel and the background pixel and asserts the is_motion output if the difference exceeds the configured threshold. It is registered on the positive clock edge and gated by the CE signal.

Line_buffer

This module implements a dual-line shift register of configurable width (default 160 pixels) that provides three row taps (tap0, tap1, tap2) corresponding to three successive rows of the image. This enables the 3x3 neighborhood access required by the Sobel kernel without full frame buffering.

sobel_calc

This module maintains a 3x3 pixel register array, updated per clock cycle from the three row taps. It computes the horizontal and vertical Sobel gradients, takes their absolute values using two's complement negation, sums them (Manhattan approximation), and outputs 0xFF if the result exceeds the Sobel threshold, otherwise 0x00.

tracker_box

This module tracks the minimum and maximum x and y coordinates of all edge pixels within a frame. It includes a pipeline readiness delay of 323 clock cycles (corresponding to two full rows plus the three-stage Sobel pipeline latency) before activating tracking. At the end of each frame, the bounding box coordinates are latched to output ports and the tracking registers are reset.

top_tracking_pipeline

This is the top-level interconnect module that instantiates and connects background_sub, line_buffer, sobel_calc, and tracker_box. It exposes the full set of input and output ports for integration with the Simulink co-simulation testbench.

2.6 Parameter Value Calculations

The key configurable parameters and their rationale are as follows:

Video Resolution: 160 x 120 pixels. This resolution was chosen to minimize simulation time while retaining sufficient spatial detail for multi-object detection in traffic scenarios.

Background Subtraction Threshold (sub_threshold): An 8-bit value (0–255). A value of 15 was used in testing, representing a moderate sensitivity that distinguishes genuine motion from minor sensor noise.

Sobel Threshold (sobel_threshold): An 11-bit value (0–2047). The maximum possible Manhattan gradient magnitude for 8-bit inputs is 4 x 255 = 1020. A threshold of approximately 100–200 was used to detect strong edges while suppressing noise.

Pipeline Delay (tracker_box): The delay of 323 cycles is derived as follows: two rows of 160 pixels each (320 cycles) to fill the line buffer, plus 3 additional cycles for the three-stage registered Sobel pipeline. This ensures the tracker only begins operating on valid, fully-computed edge data.

Minimum Object Area (MATLAB post-processing): Set to 20 pixels in the low-resolution (160x120) edge space. This filters out spurious single-pixel detections while retaining real objects of meaningful size.

Motion Threshold (MATLAB post-processing): Set to 15 gray levels. This gates the hardware edge map against a software-computed inter-frame difference, further suppressing static background edges.

CHAPTER 3: IMPLEMENTATION OF THE PROJECT

3.1 Tools and Software Used

The following software tools were used in the implementation of this project:

- MATLAB R2023b (or later): Used for video pre-processing, post-processing, and as the host environment for Simulink co-simulation.
- MATLAB Simulink: Used as the co-simulation environment for the Verilog RTL hardware pipeline, providing block-level visual integration and automated callback-driven I/O.
- Verilog HDL: Used for hardware module description of the background subtraction, line buffer, Sobel calculator, bounding box tracker, and top-level interconnect modules.
- Simulink HDL Verifier / HDL Co-simulation block (or equivalent): Used to instantiate the Verilog RTL modules within the Simulink model.
- MATLAB Image Processing Toolbox: Used for morphological operations (imdilate) and connected component analysis (regionprops) in the post-processing script.
- MATLAB Computer Vision Toolbox: Used for the insertShape and insertText functions in the output video annotation step.

3.2 Hardware Module Implementation (Verilog RTL)

All five hardware modules were implemented as synchronous RTL designs targeting positive-edge-triggered clocking with a common clock-enable signal. The design avoids asynchronous resets in all modules, using only synchronous initialization through the CE-gated always blocks.

The line_buffer module uses a pair of shift register arrays of configurable width (parameter WIDTH, default 160) to provide three-row tap access. The sobel_calc module uses a three-stage registered pipeline: the first stage loads the pixel values into the 3x3 register array from the row taps, the second stage computes the Sobel gradients, and the third stage computes the absolute values and their sum, generating the thresholded output. This pipelining enables sustained throughput of one pixel per clock cycle at full pipeline fill.

The tracker_box module addresses the pipeline latency issue by implementing a pixel_delay_count counter that inhibits tracking until 323 cycles of valid pipeline fill have elapsed. The frame counter logic increments x_c and y_c in raster scan order, resetting at frame boundaries and latching the accumulated bounding box to the output ports.

3.3 MATLAB Simulink Co-Simulation Setup

The Simulink model was configured with two callback functions defined in the Model Properties dialog:

InitFcn Callback: This script runs before simulation begins. It opens the input_video.txt file, reads all hexadecimal pixel values using textscan, converts them to double integers using hex2dec, and packages them as a Simulink.SimulationData.Dataset or timeseries structure assigned to the workspace variable input_stream. It also dynamically sets the model's StopTime to match the total number of pixel cycles in the input stream, ensuring the simulation runs for exactly the correct number of timesteps.

StopFcn Callback: This script runs after simulation completes. It retrieves the output pixel data (output_pixels) and the bounding box coordinate signals (box_min_x, box_max_x, box_min_y, box_max_y) from the simulation output object (out). The edge pixel data is written to output_edges.txt in hexadecimal format using fprintf, and the bounding box coordinate data is written to box_coordinates.txt as space-separated decimal integers, one frame per line.

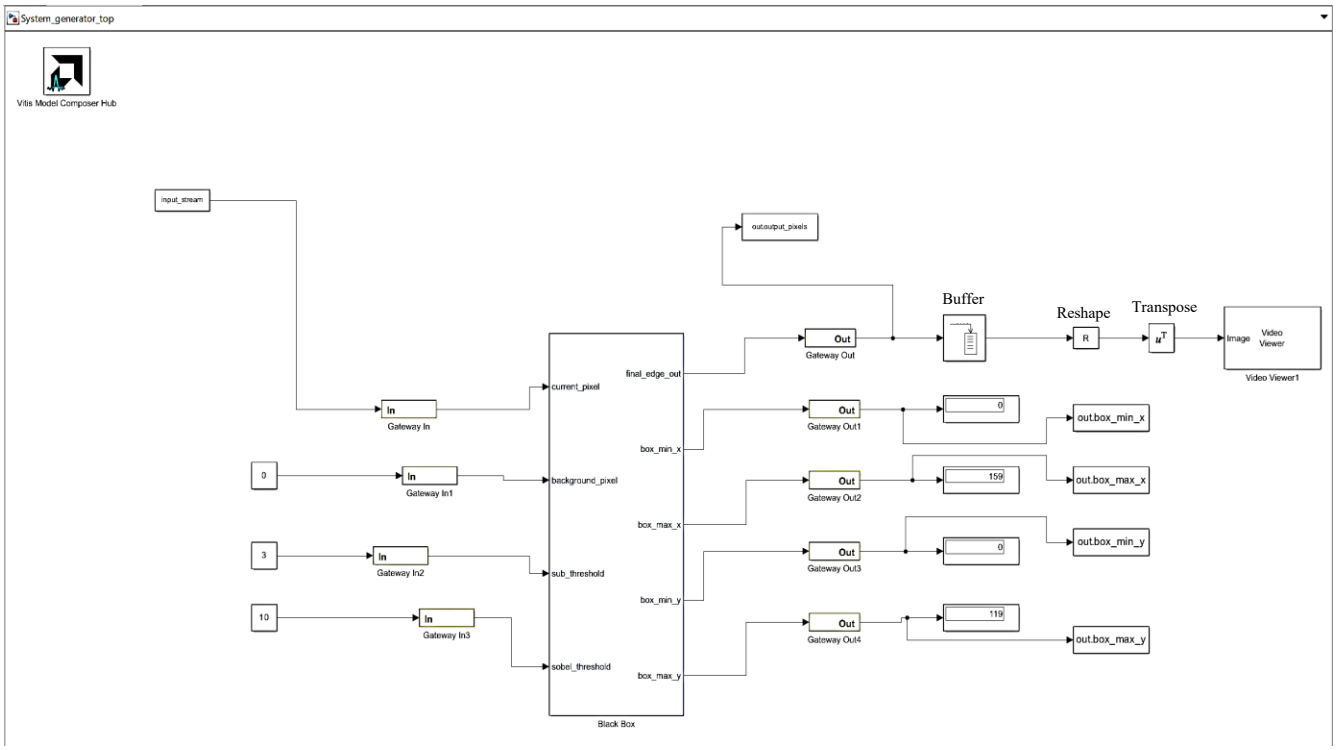

Figure 3.1: MATLAB Simulink Co-Simulation Model Showing the RTL Pipeline Integration

3.4 MATLAB Post-Processing Scripts

Two MATLAB scripts handle the software-side processing:

video_to_text.m: This script reads the input MP4 video using VideoReader, resizes each frame to 160x120 pixels using imresize, converts it to grayscale using rgb2gray, and writes all pixel values in row-major order to input_video.txt in '%02x' hexadecimal format.

multi_object_video.m: This is the primary post-processing script. It loads the hardware edge output and the original pixel stream from their respective text files and reads the original high-resolution video frame by frame using VideoReader. For each frame, it reshapes the hardware edge vector into a 160x120 matrix, computes an inter-frame motion mask by comparing the current and previous low-resolution frames against a configurable threshold, gates the hardware edge map with this motion mask, applies morphological dilation with a 3x3 structuring element to connect nearby edge fragments, and runs regionprops to identify connected regions. For each region exceeding the minimum area threshold, it scales the bounding box coordinates to the original video resolution and draws an annotated rectangle on the HD frame using insertShape. A text overlay reporting the frame number and active vehicle count is added using insertText. The annotated frames are accumulated into an output video file using VideoWriter.

3.5 Process Flow and Stages of Implementation

The complete implementation was carried out in the following stages:

Stage 1 – Hardware Module Development: Individual Verilog modules were written and functionally verified by inspection and manual trace. The background_sub, line_buffer, and sobel_calc modules were developed first, followed by the tracker_box module with its pipeline delay logic, and finally the top_tracking_pipeline interconnect.

Stage 2 – Video Pre-Processing: The video_to_text.m script was developed and executed to generate the input_video.txt pixel stream from the input MP4 file. The resulting file was verified by checking its line count against the expected value (number of frames × 160 × 120).

Stage 3 – Simulink Integration: The Simulink model was constructed with a From Workspace block sourcing the input_stream variable, connected to the HDL co-simulation block instantiating the top_tracking_pipeline module. Output signals were routed to a To Workspace block for capture. The InitFcn and StopFcn callbacks were written and attached.

Stage 4 – Co-Simulation Execution: The Simulink model was run, executing the full pixel stream through the Verilog pipeline. The output files output_edges.txt and box_coordinates.txt were generated by the StopFcn callback.

Stage 5 – Post-Processing and Output Video Generation: The multi_object_video.m script was executed on the simulation outputs and the original video to generate the final annotated 1080p output video.

CHAPTER 4: RESULTS OF THE PROJECT

4.1 Input and Output Observations

The input to the system is a traffic surveillance video (video2.mp4) containing multiple moving vehicles against a relatively static road background. The video was resized to 160x120 pixels and converted to an 8-bit grayscale pixel stream of 10,560,000 pixel samples (corresponding to 550 frames at 160x120 pixels per frame).

The Simulink co-simulation processed the full pixel stream and generated the output edge map and bounding box coordinates. The output_edges.txt file contains the same number of samples as the input, with each sample being either 0x00 (no edge) or 0xFF (edge detected). The box_coordinates.txt file contains one set of four coordinates per frame.

4.2 Output Video Analysis

The post-processing script successfully generated an annotated full-resolution output video. Moving vehicles in the scene were detected as distinct connected regions in the gated edge map and enclosed in green bounding boxes drawn on the original HD frames. The per-frame text overlay correctly reported the frame index and the number of active detected vehicles.

Multiple objects were successfully tracked simultaneously in frames where more than one vehicle was present and in motion. The motion gating step effectively suppressed detection of static roadside structures and lane markings that would otherwise generate spurious bounding boxes from the hardware edge detector.

Figure 4: Output Frame Demonstrating Multi-Object Simultaneous Tracking

4.3 Limitations

The current implementation has the following limitations:

- The single-object bounding box in the Verilog tracker_box module produces one global bounding box per frame, enclosing all detected edge pixels together. Multi-object separation is performed entirely in the MATLAB post-processing layer and is not available as a hardware output.
- The background model is static. The system uses the first frame or a pre-recorded background image as the reference, making it sensitive to illumination changes, shadows, and slow-moving objects.
- The video resolution is limited to 160x120 pixels for hardware simulation purposes, requiring an upscaling step for visualization.
- The pipeline delay compensation (323 cycles) is hardcoded based on the 160-pixel line width and the three-stage Sobel pipeline. Changes to the resolution require updating this constant.
- The co-simulation approach using text file I/O is inherently slower than real-time operation and is intended for functional verification only.

4.4 Scope for Future Work

The following enhancements are identified for future development:

- FPGA Implementation: The Verilog RTL pipeline can be synthesized and implemented on an FPGA platform (such as the Xilinx Artix-7 or Intel Cyclone series) to achieve true real-time performance. This would require interfacing with a camera input module, a pixel clock domain, and a display output interface.
- Adaptive Background Modeling: A running average background update mechanism can be implemented in hardware to accommodate gradual illumination changes and improve robustness.
- On-Chip Multi-Object Tracking: A hardware-level multi-object tracker using multiple bounding box registers or a FIFO-based connected components labeler would eliminate the current dependency on MATLAB post-processing for object separation.
- Higher Resolution Support: Parameterizing all modules for higher resolution (e.g., 640x480) and increasing the BRAM allocation for the line buffer would extend applicability to more demanding scenes.
- Deep Learning Integration: The hardware edge and motion output can serve as a pre-processing stage feeding a lightweight CNN-based classifier, enabling object class identification in addition to detection and tracking.

CONCLUSION

This project successfully demonstrated a hardware-software co-design approach for real-time motion detection and multi-object tracking. A modular, synthesizable Verilog RTL pipeline comprising background subtraction, line-buffered Sobel edge detection, and bounding box tracking was designed and functionally verified through MATLAB Simulink co-simulation.

The co-design methodology enabled an efficient division of labor: computationally intensive per-pixel operations were implemented as parallelizable hardware logic, while higher-level tasks such as multi-object separation, video reconstruction, and result visualization were handled in MATLAB. The automated callback-driven I/O in Simulink streamlined the simulation workflow significantly.

The system was validated on a real-world traffic surveillance video and produced correctly annotated output frames with bounding boxes drawn around detected moving vehicles. The results confirm the functional correctness of all hardware modules and the overall co-simulation framework.

This project lays a solid foundation for future work involving FPGA deployment, adaptive background modeling, and on-chip multi-object tracking, and demonstrates the practical value of hardware-software co-design in embedded vision applications.

REFERENCES

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[2] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed. McGraw-Hill, 2014.

[3] I. Sobel and G. Feldman, "A 3x3 isotropic gradient operator for image processing," presented at the Stanford Artificial Intelligence Project, 1968.

[4] C. Stauffer and W. E. L. Grimson, "Adaptive background mixture models for real-time tracking," in *Proc. IEEE CVPR*, Fort Collins, CO, USA, 1999, pp. 246–252.

[5] The MathWorks, Inc., *MATLAB and Simulink Documentation*. [Online]. Available: https://www.mathworks.com/help/

[6] Xilinx Inc., "FPGA-based image processing: A technical overview," White Paper WP375, 2012.

APPENDIX A: COMPLETE SOURCE CODE

The following section contains the complete source code for all Verilog RTL modules and MATLAB scripts developed in this project. The code is provided for reference and reproducibility.

A.1 background_sub.v

```
module background_sub(
    input clk,
    input ce,
    input [7:0] current_pixel,
    input [7:0] background_pixel,
    input [7:0] threshold,
    output reg is_motion
);
    reg [7:0] diff;
    always @(posedge clk) begin
     if (ce) begin
        if (current_pixel > background_pixel)
            diff <= current_pixel - background_pixel;
        else
            diff <= background_pixel - current_pixel;
        is_motion <= (diff > threshold) ? 1'b1 : 1'b0;
    end
    end
endmodule
```

A.2 line_buffer.v

```
module line_buffer #(parameter WIDTH = 160)(
    input clk,
    input ce,
    input [7:0] data_in,
    output [7:0] tap0, tap1, tap2
```

```
);
    reg [7:0] mem1 [0:WIDTH-1];
    reg [7:0] mem2 [0:WIDTH-1];
    assign tap0 = data_in;
    assign tap1 = mem1[WIDTH-1];
    assign tap2 = mem2[WIDTH-1];
    integer i;
    always @(posedge clk) begin
     if (ce) begin
       for (i = WIDTH-1; i > 0; i = i - 1) begin
          mem1[i] <= mem1[i-1];
          mem2[i] <= mem2[i-1];
       end
       mem1[0] <= tap0;
       mem2[0] <= tap1;
    end
    end
endmodule
```

A.3 sobel_calc.v

```
module sobel_calc(
    input clk,
    input ce,
    input [7:0] row0_in, row1_in, row2_in,
    input [10:0] threshold,
    output reg [7:0] edge_out
);
    reg [7:0] p00,p01,p02,p10,p11,p12,p20,p21,p22;
    reg [11:0] gx, gy, abs_gx, abs_gy, abs_g;
    always @(posedge clk) begin
     if (ce) begin
       p02<=p01; p01<=p00; p00<=row0_in;
       p12<=p11; p11<=p10; p10<=row1_in;
```

```
    p22<=p21; p21<=p20; p20<=row2_in;
        gx<=(p00+(p10<<1)+p20)-(p02+(p12<<1)+p22);
        gy<=(p00+(p01<<1)+p02)-(p20+(p21<<1)+p22);
        abs_gx<=(gx[11])?(~gx+1'b1):gx;
        abs_gy<=(gy[11])?(~gy+1'b1):gy;
        abs_g <=abs_gx+abs_gy;
        edge_out<=(abs_g>threshold)?8'hFF:8'h00;
    end
    end
endmodule
```

A.4 tracker_box.v

```
module tracker_box #(parameter WIDTH=160,parameter HEIGHT=120)(
    input clk, input ce,
    input [7:0] edge_pixel,
    output reg [7:0] box_min_x, box_max_x,
    output reg [6:0] box_min_y, box_max_y
);
    reg [7:0] x_c=0; reg [6:0] y_c=0;
    reg [7:0] mn_x=255; reg [7:0] mx_x=0;
    reg [6:0] mn_y=127; reg [6:0] mx_y=0;
    reg [9:0] pixel_delay_count=0;
    reg pipeline_ready=0;
    always @(posedge clk) begin
        if (ce) begin
            if (!pipeline_ready) begin
                if (pixel_delay_count==323) pipeline_ready<=1;
                else pixel_delay_count<=pixel_delay_count+1;
            end else begin
                if (edge_pixel==8'hFF) begin
                    if(x_c<mn_x)mn_x<=x_c; if(x_c>mx_x)mx_x<=x_c;
                    if(y_c<mn_y)mn_y<=y_c; if(y_c>mx_y)mx_y<=y_c;
                end
```

```
        if (x_c==WIDTH-1) begin
                x_c<=0;

      if (y_c==HEIGHT-1) begin
                y_c<=0;
                box_min_x<=mn_x; box_max_x<=mx_x;
                box_min_y<=mn_y; box_max_y<=mx_y;
                mn_x<=255; mx_x<=0; mn_y<=127; mx_y<=0;
              end else y_c<=y_c+1;
            end else x_c<=x_c+1;
          end
        end
      end
    endmodule
```

A.5 top_tracking_pipeline.v

```
    module top_tracking_pipeline(
      input clk, input ce,
      input [7:0] current_pixel, background_pixel,
      input [7:0] sub_threshold,
      input [10:0] sobel_threshold,
      output [7:0] final_edge_out,
      output [7:0] box_min_x, box_max_x,
      output [6:0] box_min_y, box_max_y
    );
      wire is_motion_bit;
      wire [7:0] motion_8bit,tap0,tap1,tap2;
      background_sub U_SUB(.clk(clk),.ce(ce),
        .current_pixel(current_pixel),
        .background_pixel(background_pixel),
        .threshold(sub_threshold),.is_motion(is_motion_bit));
      assign motion_8bit=is_motion_bit?8'hFF:8'h00;
      line_buffer #(.WIDTH(160)) U_BUF(.clk(clk),.ce(ce),
```

```
    .data_in(current_pixel),
        .tap0(tap0),.tap1(tap1),.tap2(tap2));
    sobel_calc U_SOB(.clk(clk),.ce(ce),

    .row0_in(tap2),.row1_in(tap1),.row2_in(tap0),
        .threshold(sobel_threshold),.edge_out(final_edge_out));
    tracker_box #(.WIDTH(160),.HEIGHT(120)) U_TRK(
        .clk(clk),.ce(ce),.edge_pixel(final_edge_out),
        .box_min_x(box_min_x),.box_max_x(box_max_x),
        .box_min_y(box_min_y),.box_max_y(box_max_y));
endmodule
```

A.6 video_to_text.m

```
video_path = 'D:\minorproj2\videos\video2.mp4';
output_txt = 'D:\minorproj2\input_video.txt';
width = 160; height = 120;
v = VideoReader(video_path);
fileID = fopen(output_txt, 'w');
while hasFrame(v)
    frame = readFrame(v);
    frame_resized = imresize(frame, [height, width]);
    gray_frame = rgb2gray(frame_resized);
    gray_transposed = gray_frame';
    fprintf(fileID, '%02x\n', gray_transposed(:));
end
fclose(fileID);
```

A.7 multi_object_video.m

```
clear; clc;
% CHANGE THIS to the exact filename of your original high-quality video
HIGH_RES_VIDEO = 'D:\minorproj2\videos\video2.mp4';

MIN_OBJECT_AREA = 20;
MOTION_THRESHOLD = 15;

if ~exist('output_edges.txt', 'file') || ~exist('input_video.txt', 'file')
```

```
error('Missing required simulation text files!');
end
if ~exist(HIGH_RES_VIDEO, 'file')

error('High-resolution video file not found!');
end


% Load Hardware Data (Safely split to avoid indexing error)
fprintf('Loading hardware edge map...\n');
fid = fopen('output_edges.txt', 'r');
raw_edges = textscan(fid, '%s');
fclose(fid);
edge_pixels = double(hex2dec(raw_edges{1}));

fprintf('Loading original low-res reference stream...\n');
fid = fopen('input_video.txt', 'r');
raw_in = textscan(fid, '%s');
fclose(fid);
display_pixels = double(hex2dec(raw_in{1}));

hr_reader = VideoReader(HIGH_RES_VIDEO);

lr_height = 120; lr_width = 160;
pixels_per_frame = lr_height * lr_width;
total_frames = min(floor(length(edge_pixels) / pixels_per_frame), hr_reader.NumFrames);

hr_width = hr_reader.Width; hr_height = hr_reader.Height;
scale_x = hr_width / lr_width;
scale_y = hr_height / lr_height;

% Initialize HD Writer
v = VideoWriter('final_1080p_multi_tracked.mp4', 'MPEG-4');
v.FrameRate = hr_reader.FrameRate;
open(v);

prev_frame_matrix = zeros(lr_height, lr_width);

fprintf('Compiling %d HD frames with scaled multi-object tracks...\n', total_frames);

for f = 1:total_frames
frame_start = (f-1) * pixels_per_frame + 1;
frame_end = f * pixels_per_frame;

% Pull the original HD frame
img_hr = read(hr_reader, f);

disp_matrix = double(reshape(display_pixels(frame_start:frame_end), [lr_width, lr_height])');
edge_matrix = reshape(edge_pixels(frame_start:frame_end), [lr_width, lr_height])';
binary_edges = edge_matrix > 128;

if f > 1
motion_mask = abs(disp_matrix - prev_frame_matrix) > MOTION_THRESHOLD;
else
motion_mask = zeros(lr_height, lr_width);
end
prev_frame_matrix = disp_matrix;

gated_tracked_edges = binary_edges & motion_mask;
se = strel('square', 3);
gated_tracked_edges = imdilate(gated_tracked_edges, se);

stats = regionprops(gated_tracked_edges, 'BoundingBox', 'Area');
objects_in_frame = 0;
```

```
for k = 1:length(stats)
if stats(k).Area > MIN_OBJECT_AREA
bbox = stats(k).BoundingBox;

% Scale boxes to HD layer

hr_bbox = [bbox(1)*scale_x, bbox(2)*scale_y, bbox(3)*scale_x, bbox(4)*scale_y];


img_hr = insertShape(img_hr, 'Rectangle', hr_bbox, ...
'Color', 'green', 'LineWidth', 4, 'SmoothEdges', true);
objects_in_frame = objects_in_frame + 1;
end
end

img_hr = insertText(img_hr, [20 20], sprintf('Frame: %d | Active Vehicles Detected: %d', f, objects_in_frame), ...
'FontSize', 24, 'BoxColor', 'black', 'TextColor', 'green');

writeVideo(v, img_hr);
end

close(v);
```

A.8 top_tracking_pipeline_config.m

```
function top_tracking_pipeline_config(this_block)

 this_block.setTopLevelLanguage('Verilog');

 this_block.setEntityName('top_tracking_pipeline');


 this_block.tagAsCombinational;

 this_block.addSimulinkInport('current_pixel');
 this_block.addSimulinkInport('background_pixel');
 this_block.addSimulinkInport('sub_threshold');
 this_block.addSimulinkInport('sobel_threshold');

 this_block.addSimulinkOutport('final_edge_out');
 this_block.addSimulinkOutport('box_min_x');
 this_block.addSimulinkOutport('box_max_x');
 this_block.addSimulinkOutport('box_min_y');
 this_block.addSimulinkOutport('box_max_y');

 final_edge_out_port = this_block.port('final_edge_out');
 final_edge_out_port.setType('UFix_8_0');

 box_min_x_port = this_block.port('box_min_x');
 box_min_x_port.setType('UFix_8_0');


box_max_x_port = this_block.port('box_max_x');
 box_max_x_port.setType('UFix_8_0');

 box_min_y_port = this_block.port('box_min_y');
 box_min_y_port.setType('UFix_7_0');
```

```
 box_max_y_port = this_block.port('box_max_y');
 box_max_y_port.setType('UFix_7_0');

if (this_block.inputTypesKnown)

if (this_block.port('current_pixel').width ~= 8)
     this_block.setError('Input data type for port "current_pixel" must have width=8.');
   end

   if (this_block.port('background_pixel').width ~= 8)
    this_block.setError('Input data type for port "background_pixel" must have width=8.');
   end

   if (this_block.port('sub_threshold').width ~= 8)
    this_block.setError('Input data type for port "sub_threshold" must have width=8.');
   end

   if (this_block.port('sobel_threshold').width ~= 11)
    this_block.setError('Input data type for port "sobel_threshold" must have width=11.');
   end

 end  % if(inputTypesKnown)

 if (this_block.inputRatesKnown)
    setup_as_single_rate(this_block,'clk','ce')
  end  % if(inputRatesKnown)

 uniqueInputRates = unique(this_block.getInputRates);

this_block.addFile('C:/Users/Mayur/Object_Tracker_V2/Object_Tracker_V2.srcs/sources_1/imports/Background_Substraction/top_tracking_pipeline.v');

this_block.addFile('C:/Users/Mayur/Object_Tracker_V2/Object_Tracker_V2.srcs/sources_1/imports/Background_Substraction/background_sub.v');

this_block.addFile('C:/Users/Mayur/Object_Tracker_V2/Object_Tracker_V2.srcs/sources_1/imports/Background_Substraction/line_buffer.v');

this_block.addFile('C:/Users/Mayur/Object_Tracker_V2/Object_Tracker_V2.srcs/sources_1/imports/Background_Substraction/sobel_calc.v');

this_block.addFile('C:/Users/Mayur/Object_Tracker_V2/Object_Tracker_V2.srcs/sources_1/imports/Background_Substraction/tracker_box.v');
return;
function setup_as_single_rate(block,clkname,cename)
  inputRates = block.inputRates;
  uniqueInputRates = unique(inputRates);
  if (length(uniqueInputRates)==1 && uniqueInputRates(1)==Inf)
    block.addError('The inputs to this block cannot all be constant.');
    return;
  end
  if (uniqueInputRates(end) == Inf)
     hasConstantInput = true;
     uniqueInputRates = uniqueInputRates(1:end-1);
  end
  if (length(uniqueInputRates) ~= 1)
    block.addError('The inputs to this block must run at a single rate.');
```

```
    return;
  end
  theInputRate = uniqueInputRates(1);
  for i = 1:block.numSimulinkOutports
     block.outport(i).setRate(theInputRate);
  end
  block.addClkCEPair(clkname,cename,theInputRate);
  return;

function arrayHDLType = convertArrayType(inArr)
arrayHDLType = '';
for i=1:length(inArr)
    if (i == 1)
        arrayHDLType = [arrayHDLType '(' num2str(inArr(i))];
    elseif (i == length(inArr))
        arrayHDLType = [arrayHDLType ',' num2str(inArr(i)) ')'];
    else
        arrayHDLType = [arrayHDLType ',' num2str(inArr(i))];
    end
end
```